

Function with Default Parameters

In C++ programming, we can provide default values for function parameters.

1. If a function with default arguments is called without passing arguments, then the default parameters are used.
1. However, if arguments are passed while calling the function, the default arguments are ignored.

Example:

```
int sum ( int a = 0, int b = 10, int c =5)
{
return a+b+c;
}
```

y = sum(1,6); // returns 12

y = sum(); // returns 15

y = sum(6,1); // returns 12

y = sum (2,3,4); // returns 9

Can be provided for trailing arguments only

Examples:

```
int f1 ( int a, int b = 1, int c = 5); // OK
```

```
int f2 (int a = 2, int b = 1, int c); // Error
```

```
int f3 (int a = 2, int b, int c = 5); // Error
```

Function Call

The two types of function call:

- Call by value
- Call by reference

Arguments can generally be passed in one of the two ways:

Sending the values of the arguments

In this method, value of each of the actual arguments in the calling function is copied into corresponding formal arguments of the called function. With this method changes made to the formal arguments in the called function have no effect on the values of the actual arguments.

Sending the address of the arguments

The address of the actual arguments in the calling function are copied into formal arguments of the called function. This

Function Overloading

Function Overloading

1. Overloading refers to the use of same thing for different purpose.
2. C++ permits overloading of functions.
3. same function name may be used to create functions that perform a variety of different tasks.
4. The correct function to be invoked is determined by checking the number and type of arguments but not on the function type.

For example an overloaded **add()** function handles different types of data as shown below:

// **Declarations**

int add(int a,int b); // prototype 1

int add(int a, int b, int c); // prototype 2

double add(double x, double y); // prototype 3

double add (int p, double q); // prototype 4

double add(double p, int q); // prototype 5

//Function calls

cout<<add(5,10); //uses prototype 1

cout<<add(15,10.0); // uses prototype 2

cout<<add(12.5,7.5); // uses prototype 3

cout<<add(5,10,15); // uses prototype 4

cout<<add(0.75,5); // uses prototype 5

Function Templates

Suppose we want to write a function that returns the minimum of two numbers. ordinarily this function would be written for a particular data type. For example:

```
int min(int a, int b)
{
return (a<b)?a:b;
}
```


If we want to find a minimum of two long *ints*-we would be required to write a completely new function . Similarly to find minimum of two floats or two doubles or two chars it is required to write separate versions of the same function. These are given below:

//min for ints

```
int min(int a,int b)
{ return (a<b)?a:b;
}
```

//min for longs

```
long min (float a, float b)
{
return (a<b)?a:b;
}
```

Have we gained anything by writing these overloaded functions?

This results into following disadvantages:

1. Rewriting the same function body over and over for different types is time consuming.
2. Program consumes more disk space.

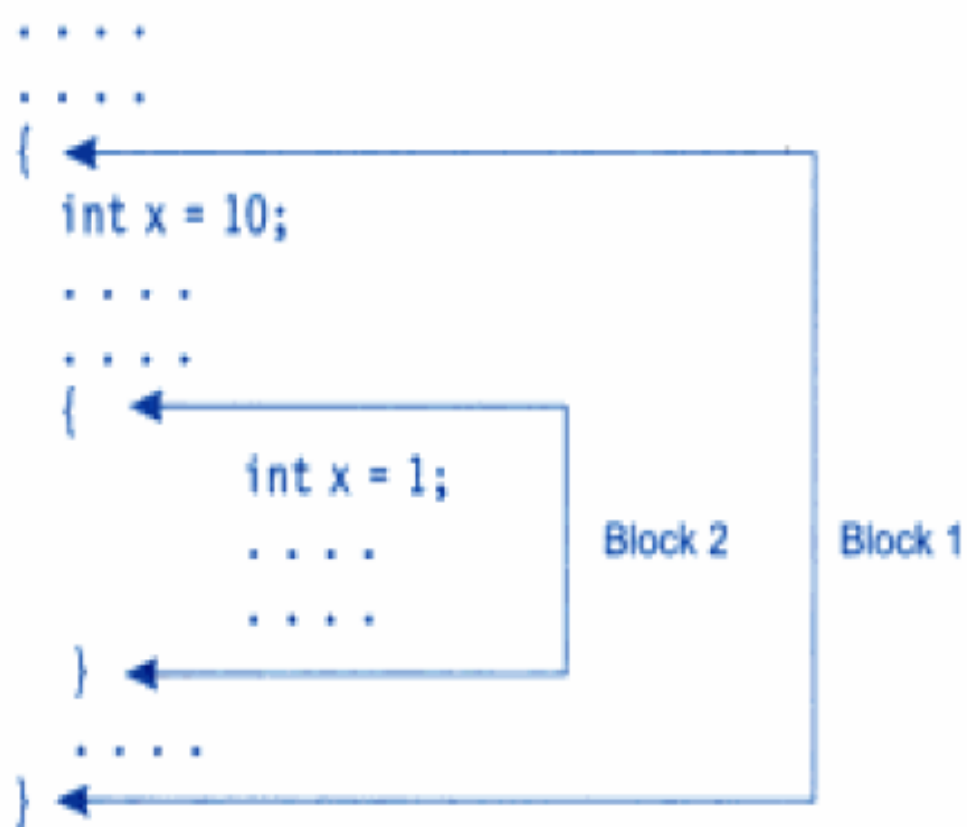
It would be nice if we could write such a function just once, and make it work for many different data types. This is exactly what function templates do for us.

The following program shows how to write the min() function as a template, so that it will work with standard type. We have invoked this function from main() for different data types.

```
#include<iostream>
using namespace std;
template<class T>
T min(T a, T b)
{
    return (a < b) ? a : b;
}
int main()
{
    int i = 10, j = 20;
    cout << min(i, j) << endl;
    cout << min(3.14, -6.28)<<endl;
    cout << min('A', 'Z')<<endl;
    cout << min(1.1, 1.11) << endl;
    system("pause");
    return 0;
}
```

Unary Scope Resolution Operator

1. Same variable name can be used to have different meanings in different blocks.
2. The scope of variable extends from the point of its declaration till the end of the block containing the declaration.
3. A variable declared inside a block is said to be local to that block.



1. The two declarations of x refer two different memory locations containing different values. Statements in the second block cannot refer to the variable x declared in the final block , and vice versa. Blocks in C++ are often nested.
2. Block 2 is contained in block 1. note that a declaration in an inner block hides a declaration of the same variable in an outer block and, therefore , each declaration of x causes it to refer to a different data object. Within inner block, the variable x will refer to the data object declared therein.
3. C++ resolves this problem by introducing a new operator :: called scope resolution operator. This can be used to uncover a hidden variable. It takes the following form:
 :: variable_name
4. This operator allows access to the global version of a variable.

```

#include<iostream>
using namespace std;
int m = 10; //global m
int main()
{
    int m = 20; //m declared local to main
    {

        int k = m;
        int m = 30; // m declared again
        cout << "we are in the inner block" << endl;
        cout << " k = " << k << endl;
        cout << " m = " << m << endl;
        cout << " : : m = " << ::m << endl;
    }
    cout << " we are in outer block " << endl;
    cout << " m = " << m << endl;
    cout << ":: m= " << ::m << endl;

    system("pause");
    return 0;
}

```